PassCluster: A New Segmented Password Cracking Model

HU XIONG, University of Electronic Science and Technology of China, Chengdu, China
KAIWEN XING, University of Electronic Science and Technology of China, Chengdu, China
XUDONG YANG, University of Electronic Science and Technology of China, Chengdu, China
JINCHENG LI, University of Electronic Science and Technology of China, Chengdu, China
WULONG FAN, University of Electronic Science and Technology of China, Chengdu, China
ABUBAKER WAHABALLA, Arab East Colleges, Riyadh, Saudi Arabia
KUO-HUI YEH*, National Yang Ming Chiao Tung University, Hsinchu, Taiwan and National Dong Hwa University, Hualien, Taiwan

Passwords remain a widely deployed component of digital authentication because of their low cost and broad compatibility, but users' weak or reused passwords make them vulnerable to attacks. Advancing password-cracking research is vital to assess password strength and improve cybersecurity. Current password models exhibit biased structural segmentation: Markov models are overly fine-grained, leading to weak contextual dependencies, while PCFG models are too coarse to capture detailed features. Although existing segmentation-based models have to some extent balanced the granularity issue of password processing units, they still lack effective partitioning strategies, leading to suboptimal overall modeling performance. To address this problem, this paper proposes PassCluster, a model that clusters the characters within passwords and segments them based on the clustering results. It then utilizes an LSTM network to model sequential dependencies between segments and learns a left-to-right conditional probability distribution over segment sequences, enabling password generation by sampling segments in order. This approach enables more reasonable password segmentation, allowing the model to generate more accurate password guesses. Extensive experiments on eight real-world datasets demonstrate that the proposed approach improves guessing accuracy by 27.0% and 85.5% in intra-site and cross-site scenarios, respectively. Moreover, under the same number of unique password guesses, PassCluster exhibits strong performance and maintains competitive effectiveness. Furthermore, a real-time Password Strength Meter is developed based on this model to assist users in constructing stronger passwords, underscoring the importance of incorporating fine-grained segmentation and probabilistic modeling in password security evaluation.

CCS Concepts: • **Security and privacy → Authentication**.

1 Introduction

Passwords remain a widely deployed component of contemporary authentication ecosystems, although they increasingly coexist with multi-factor, biometric, and passwordless mechanisms. Their continued use is associated with low deployment cost, broad compatibility, scalability, and ease of replacement after compromise [10, 47].

Authors' Contact Information: Hu Xiong, University of Electronic Science and Technology of China, Chengdu, Sichuan, China; e-mail: xionghu@uestc.edu.cn; Kaiwen Xing, University of Electronic Science and Technology of China, Chengdu, Sichuan, China; e-mail: X858988337@gmail.com; Xudong Yang, University of Electronic Science and Technology of China, Chengdu, Sichuan, China; e-mail: swithunyang918@gamil.com; Jincheng Li, University of Electronic Science and Technology of China, Chengdu, Sichuan, China; e-mail: ssqq42852@gmail.com; Wulong Fan, University of Electronic Science and Technology of China, Chengdu, Sichuan, China; e-mail: fanwulon@163.com; Abubaker Wahaballa, Arab East Colleges, Riyadh, Riyadh Province, Saudi Arabia; e-mail: wahaballah@arabeast.edu.sa; Kuo-Hui Yeh, National Yang Ming Chiao Tung University, Hsinchu, Taiwan and National Dong Hwa University, Hualien, Taiwan; e-mail: khyeh@nycu.edu.tw.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2471-2574/2026/10-ART
https://doi.org/10.1145/3859248

Just Accepted

This persistence should not be interpreted as evidence that passwords are universally irreplaceable: alternative mechanisms can reduce or eliminate passwords in particular settings, but password-based authentication remains relevant during the ongoing transition. Its security therefore continues to have substantial practical consequences. For example, the 2025 Verizon Data Breach Investigations Report analyzed 12,195 confirmed breaches and identified credential abuse as the leading initial-access vector, accounting for 22% of the breaches studied [36]. In particular, users' tendency to select weak or reused passwords exposes password-based systems to dictionary, brute-force, password-guessing, and credential-stuffing attacks [6, 19, 34, 38]. Consequently, understanding password characteristics and improving password-cracking models remain important for evaluating password strength and enhancing cybersecurity [16, 20, 41].

Traditional dictionary attacks and brute-force methods rely heavily on the coverage of their dictionaries, which limits the variety of passwords they can generate. Contemporary password-generation/cracking models can be broadly categorized into two classes. The first class operates at the character level (character-level), exemplified by Markov-based approaches [19] and most neural-network models (e.g., FLA [20]). These models treat individual characters as the basic processing unit and thus struggle to capture semantic and higher-level structural information in passwords. The second class is template-based (template-level), represented by Probabilistic Context-Free Grammars (PCFG) [42], which decompose passwords into fields of character classes such as letters (L), digits (D), and special symbols (S) and model composition at that level. However, such template segmentation is often too coarse, for example, labeling "AppleTreeHouse" as "L14" ignores its internal semantic segments (Apple / Tree / House). In summary, character-level models are fine-grained but lack semantic awareness, while template-level models incorporate structural information yet are overly coarse, neither adequately captures the compositional patterns of real world passwords.

To address the limitations of existing approaches, researchers have proposed various segment-based password cracking models aimed at identifying more appropriate processing units. One strategy involves semantic segmentation through natural language processing (NLP), as seen in models like Semantic_PCFG [35] and SE#PCFG [40]. Another strategy is based on statistical analysis of character transition patterns, such as in WordPCFG [3] and CKL_PCFG [44]. However, both approaches face challenges. NLP-based models struggle with adaptation, as passwords are typically much shorter than natural language texts (e.g., 8–12 characters), resulting in fragmented semantics that cannot be easily handled by conventional language corpora. For instance, the password "password123" might be incorrectly segmented into "pass" and "word123", failing to recognize its overall meaning. Moreover, passwords often contain nonstandard expressions (e.g., "4ever") that differ significantly from natural language, making semantic parsing difficult. Similarly, statistical segmentation models also have inherent limitations. These models typically emphasize transitions between adjacent characters, while overlooking potential relationships among non-adjacent characters that belong to the same semantic unit. For instance, consider the password "astrongpassword". In segmentation methods like CKL_PCFG, the password may be segmented into fragments such as "as", "tr", "ong" and "password". This happens because the model relies heavily on local adjacency statistics learned from the training set. If certain character transitions (e.g., "a" followed by "s") occur more frequently than others, the model tends to assemble the characters at those positions, even when the resulting segmentation violates natural semantic boundaries. As a result, the segmentation process captures statistical co-occurrence patterns rather than coherent semantic structures, which weakens the model's ability to understand and generate complex passwords effectively. These shortcomings reveal a fundamental gap: existing segmentation methods rely either on predefined linguistic knowledge or local statistical dependencies, but neither can flexibly discover the latent structural communities that naturally exist within password datasets.

To address these challenges, we propose a novel password cracking model called PassCluster, which integrates the Louvain algorithm with an Long Short-Term Memory (LSTM) network for password segmentation and sequence analysis. The Louvain algorithm is a community detection clustering method that groups password elements by maximizing modularity. This process uncovers implicit structural clusters without relying on external

Just Accepted

linguistic rules. It also has low computational complexity, which makes it suitable for large password datasets, and it identifies segments that are similar in semantics or structure. Furthermore, as a modularity-based clustering algorithm, Louvain pays greater attention to the internal character composition of frequently occurring substrings rather than merely modeling local adjacency relationships between neighboring characters. This enables the model to capture globally coherent structural patterns within passwords, thereby reducing unreasonable fragmentations caused by excessive reliance on adjacent character transitions. Consequently, semantically meaningful segments can be preserved more effectively during password segmentation. On this basis, we apply an LSTM at the segment level to capture dependencies among segment sequences. The LSTM learns sequential patterns conditioned on historical context and generates password candidates accordingly. This combination connects global structural information from Louvain with sequential regularities learned by the LSTM, which improves the model's ability to represent complex password structures. Compared with existing methods, the proposed model provides stronger interpretability. The Louvain algorithm captures relationships among different characters or substrings within a password and supports more appropriate structural categorization. The LSTM generation process keeps password derivation grounded in reasonable segment units and uses contextual information from known segments to predict the most likely subsequent segment, which improves generation accuracy and coverage.

Furthermore, we introduce a Password Strength Meter (PSM) based on this model, which provides real-time feedback on password strength and alerts users to replace weak segments.

We summarize our main contributions as follows:

- We introduce the Louvain algorithm for community detection to effectively segment passwords, ensuring that the resulting segments maintain stronger semantic coherence and connectivity. This novel segmentation approach enhances the logical structure of password clustering, improving the efficiency of password pattern analysis.
- We propose PassCluster, a novel password security model that integrates password segmentation with LSTM based sequence modeling. Evaluations on eight real-world datasets demonstrate that PassCluster significantly outperforms existing mainstream models in password cracking, offering improved security in real-world scenarios.
- PassCluster exhibits a significant advantage in generating unique password guesses, maintaining strong cracking performance under the same number of distinct attempts. This characteristic is highly valuable in real-world applications, where avoiding redundant guesses directly improves efficiency and practical impact.
- We have developed a PSM, which provides real-time security feedback using the password model from this study, helping users create more secure passwords.

2 Preliminaries

2.1 Mainstream Password Cracking Model

2.1.1 Dictionary-based password cracking model. Dictionary-based password cracking methods leverage existing password datasets to learn and construct rule-based dictionaries, which are then used to generate candidate passwords. The effectiveness of this approach largely depends on the richness of the password datasets and the accuracy of the rule dictionaries [14]. Hashcat [33] and JTR [29] are typical examples of such methods. Since the generation of candidate passwords relies solely on the initial password set and rule dictionaries, the maximum number of guesses for these models is the product of the size of the password set and the number of rules, meaning they can only generate a limited number of candidate passwords. Furthermore, password creation habits vary significantly across platforms, languages, and user groups, making it challenging for a fixed rule set to adapt effectively to diverse password datasets. As a result, dictionary-based methods usually suffer from limited flexibility and reduced generalization capability in real-world cross-site password cracking scenarios. To address

Just Accepted

these limitations, AdaMs_Hashcat [27] introduced adaptive and dynamic dictionary attacks that leverage deep neural networks to model the compatibility between dictionary words and mangling rules, thereby improving the flexibility and effectiveness of dictionary-based password cracking in real-world scenarios.

2.1.2 Statistics-based password cracking model. These models typically regard passwords as sequences of smaller elements and calculate the overall password probability by performing statistical analysis on these elements. Based on the different smallest processing unit, these models can be categorized into two major types: character-level models, template-level models and segment-level models.

Character-level models treat characters as the smallest unit and generate passwords character by character. A typical example is the Markov model [21], which predicts the next possible character based on preceding characters. Subsequent studies have introduced several improvements, such as smoothing and regularization techniques to address overfitting [19], and the design of Markov enumerators to enable probability-based password generation [9]. Another notable character-level model is FLA [20], which leverages neural networks to learn from large-scale password datasets, automatically extracting features and information to build generative models. In recent years, several efficient character-level password guessing approaches have emerged. For instance, VAEPass [45] proposed a lightweight password generation model based on a variational autoencoder (VAE), which significantly improved generation efficiency and accuracy by incorporating token-level modeling, while also substantially reducing training time and resource consumption. PassFlow [25] introduced a reversible generative flow-based method for password guessing, which outperformed existing GAN- and VAE-based approaches even with significantly less training data. RFGuess [39] leveraged a random forest-based re-encoding of password characters, achieving performance comparable to or better than deep learning models across various attack scenarios, particularly in low-data environments. PassGPT [31] introduced an autoregressive GPT-2-based password generation model trained on password leaks, which explicitly models password probability distributions and enables character-level guided generation under arbitrary constraints. PassLLM [48] proposed a LoRA-based fine-tuning framework for decoder-only large language models, supporting both trawling and targeted password guessing scenarios while improving generation efficiency through tailored inference algorithms and model distillation.

Template-level models segment passwords into templates according to predefined rules, and the probability of a password is modeled as the product of the probabilities of its template and its components. The most representative model in this category is PCFG [42], which extracts structural patterns and probability distributions from letters, digits, and special characters, and generates candidate passwords based on these structures.

Segment-level models operate passwords with a granularity between the above two. These models first divide a password into segments using predefined rules or statistical methods, then learn the distribution of these segments to guide password prediction. As shown in Table 1, prior work has proposed structural segmentation methods, such as incorporating predefined dictionaries like corpora, keyboard patterns and email addresses [35, 40], or using statistical approaches like degrees of freedom, cohesion, and BPE [3, 43, 44]. In addition to the aforementioned classic models, several recent studies have introduced enhancements from different perspectives.

2.1.3 Emerging Password Guessing, Evaluation and Defense Paradigms. Recent studies have expanded password research beyond conventional static, whole-password generation. Guan and Chen [15] propose an online verification scheme that uses a statistical model of legitimate-user and attacker behavior together with the entropy of entered passwords to identify online guessing attempts, providing an additional layer of protection even when users select common passwords. MoPE [8] exploits password structures by constructing specialized expert models for different structural patterns and uses a lightweight gating mechanism to select suitable experts during generation. CoT-DPG [37] studies dynamic password guessing, in which feedback obtained during guessing is used to incrementally adapt multiple complementary models through co-training. Mask password guessing [49] considers attackers who obtain partial information, such as password length or individual characters, through

Just Accepted

Table 1. Comparison of Mainstream Segmented Model and PCFG Technologies

Types of segmentation	Approach	Core Techniques
Based on semantics or rules	PCFG [42]	Letters, digits and symbols patterns
	Semantic PCFG [35]	POS tagging and semantic mapping
	NPC [17]	Keyboard patterns, multi-word combinations and repetition structures
	SE#PCFG [40]	43 Semantic and contextual features
Statistical relationship between adjacent characters	WordPCFG [3]	Cohesion and freedom
	CKL_PCFG [44]	Byte pair encoding
Adaptively categorize characters	PassCluster	Louvain community detection algorithm

side channels, and combines such information with PII and previously leaked passwords. From the defensive perspective, Pickering et al. [30] investigate foundation LLMs for password-strength estimation and user feedback, showing their potential for recognizing semantic weaknesses while also identifying limitations in evaluating complex passwords and generating secure recommendations.

2.2 Datasets

The eight datasets used in this paper include four from Chinese-speaking platforms and four from English-speaking platforms (see Table 2). Specifically, CSDN [5] is the largest technical community website in China, gathering a vast number of software developers and IT professionals. Netease [23] is the email service platform of NetEase, which has a broad user base in China and is favored for its stability and convenience. Dodonew [7] is an influential e-commerce website that provides users with a wide range of shopping choices and convenient transaction experiences. 7k7k [1] is a popular Chinese online gaming platform that offers a wide variety of casual games and is especially favored by younger audiences. Rockyou [32] is a social entertainment and tools website, primarily focused on social media platforms, helping users create and share interesting content. Gmail [13] is a widely used email service developed by Google, known for its security, large storage capacity, and convenient integration with other Google services. Neopets [22] is a virtual pet website where users can create and care for digital pets while enjoying various games and community activities. Clixsense [4] is an online task and survey platform that offers users opportunities to earn money by completing simple tasks and participating in surveys, and is widely popular among global part-time earners.

To prevent further harm to victims caused by data leaks, we removed all personal information, such as names and email addresses, and only retained the password data. Additionally, considering real-world website limitations on user passwords, we retained only passwords composed of the 94 printable ASCII characters excluding the space character and excluded passwords shorter than 6 characters or longer than 20 characters.

2.3 Louvain Algorithm

The Louvain algorithm [2] is an efficient community detection algorithm based on modularity optimization, aiming to discover the multi-level community structure of the network. The algorithm is divided into two stages:

Just Accepted

Table 2. Leaked password datasets

Dataset	Language	Web Service	Leaked Time	Total Passwords	Unique Passwords
CSDN	Chinese	IT forum	Dec. 2011	4,502,972	2,907,975
Netease	Chinese	Email	Oct. 2015	233,379,434	12,120,997
Dodonew	Chinese	E-commerce	Dec. 2011	16,258,891	8,864,310
7k7k	Chineses	Online games	Dec. 2011	19,138,452	2,865,573
Rockyou	English	Social forum	Dec. 2009	32,381,998	14,207,429
Gmail	English	Email	Sep. 2014	4,691,609	3,325,674
Neopets	English	Virtual-pet	Jul. 2016	67,672,205	27,474,769
Clixsense	English	Paid surveys	Sep. 2016	2,222,045	1,628,018

local optimization and community aggregation. In the local optimization stage, each node is initially regarded as an independent community. During the iteration process, each node attempts to move to the neighboring community and selects the community that maximizes the modularity increment ΔQ. The ΔQ quantifies the increase in overall network modularity resulting from moving a node to a different community, serving as a measure of whether the new partitioning better reflects the underlying community structure. The formula for the modularity increment is:

$$\Delta Q = \left[\frac{\sum_{in} + 2\sum_{i,in}}{2m} - \left(\frac{\sum_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\sum_{in}}{2m} - \left(\frac{\sum_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right] \tag{1}$$

where $\sum_{\text{in}}$ is the sum of internal edges weights in the new community, $\sum_{\text{tot}}$ is the sum of total degrees of all nodes in the new community, $\sum_{\text{i, in}}$ is the sum of the edges weights of node i and all other nodes in the new community, k_i is the degree of node i, and m is the total number of edges. If ΔQ is positive, then moving node i to a new community improves the overall modularity, and the algorithm performs the move, otherwise it remains unchanged. After the local optimization is completed, the community aggregation stage is entered: each community is regarded as a supernode, a simplified network is constructed, and the above process is repeated until the modularity no longer increases. Louvain algorithm has excellent scalability and accurate modularity optimization, can efficiently deal with large-scale networks, and reveal hierarchical community structure, and is suitable for large data set analysis [2, 12, 18].

3 PassCluster: Louvain algorithm based password guessing model

3.1 Model Construction

To ensure password security, we propose a password cracking model with a relatively balanced granularity, which segments passwords into more reasonable units for probability estimation. As shown in Figure 1, the Louvain algorithm is first applied to adaptively cluster characters in the training set, learning both the number of categories and their structure. Then, passwords are segmented accordingly, and a segment level probabilistic model is constructed on the resulting segment sequences. After that, this work adopts an LSTM to perform left-to-right prediction of next segment, thereby computing the log probabilities of passwords in the test set. Finally, based on model generation and sampling results, the Monte Carlo method is used to simulate the number of guesses required for the test passwords. This approach preserves the interpretability of segment based modeling

Just Accepted

while better capturing sequential dependencies between segments, which aligns with users typical password creation patterns.

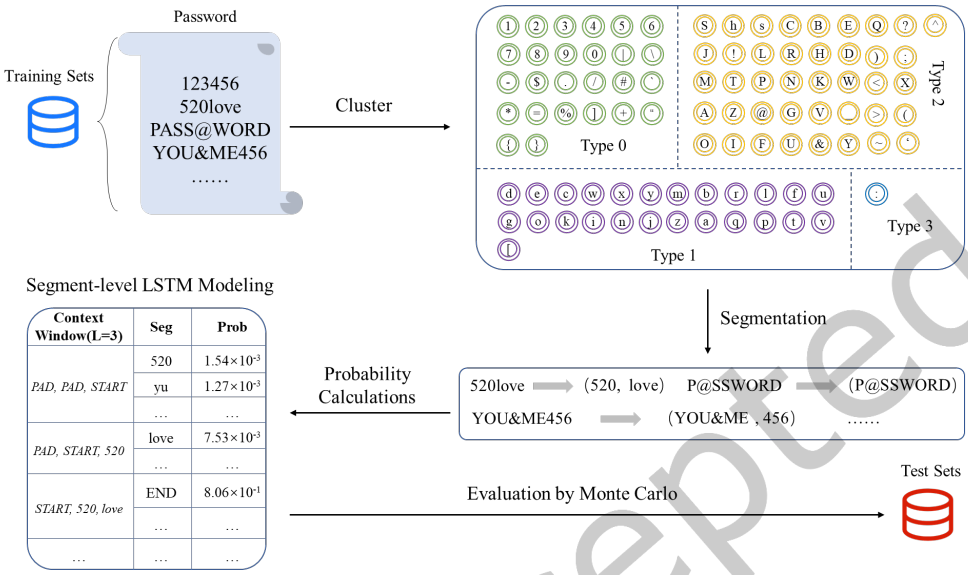


Fig. 1. Construction of PassCluster model. Based on real user password samples, we first cluster characters to construct a segmented probabilistic model. Subsequently, new passwords are generated via Monte Carlo simulation, and their predictive performance is evaluated against the test set.

3.1.1 Clustering. We first construct a character co-occurrence graph from all passwords in the training set and then perform clustering on the characters based on their connection patterns. Specifically, the 94 printable ASCII characters are treated as graph nodes. For each password, edges are added between adjacent characters appearing in the password. For example, the password "520love" generates adjacent character pairs such as (5, 2), (2, 0), (0, l), (l, o), (o, v) and (v, e). If the same pair of adjacent characters appears multiple times across different passwords, the corresponding edge weight is increased accordingly. After processing all passwords in the training set, all character relationships are merged into a single global character graph, where nodes represent ASCII characters and weighted edges represent the frequency of adjacency between characters in passwords. Based on this graph, we apply the Louvain community detection algorithm to partition the 94 ASCII characters into several clusters according to their co-occurrence connectivity. Characters that frequently appear together or consecutively in passwords are therefore grouped into the same category. As illustrated in Figure 1, for the password "520love", the characters "5", "2" and "0" are assigned to one cluster (type 0), while "l", "o", "v" and "e" are assigned to another cluster (type 1). In this way, the clustering process divides the 94 ASCII characters into different categories according to the statistical adjacency patterns learned from the entire training set.

3.1.2 Segmentation. Starting from the first character of the password, we check whether it belongs to the same cluster as the previous character (for the first character, we assume a virtual "start" character before it, which does not belong to any category). If they belong to the same cluster, they are combined to form an initial segment. Then, we examine the next character and determine whether it belongs to the same category as the current segment. If it does, the character is added to the current segment; if not, the expansion of the current segment is

Just Accepted

stopped, and it is considered that a complete segment has been found. For example, according to the clustering results, the characters “5”, “2”, and “0” are classified into category 0, while the characters “l”, “o”, “v”, and “e” are classified into category 1. Therefore, the password “520love” is divided into two segments: “520” and “love”. This means that the segmentation is performed based on the character classification results obtained from clustering, dividing the password into a combination of several segments.

3.1.3 LSTM-based model training. Next, it is necessary to count the frequency of occurrences of all prefixes and their corresponding succeeding segments (or suffixes). Specifically, we build a segment vocabulary from the training set and introduce special tokens <START>, <END>, and <PAD>. The training task is formulated as next-token prediction: given a context beginning with <START> and followed by previously generated segments, the model predicts the next segment, and finally predicts <END> to terminate the sequence. To keep the inference pattern consistent with our previous left-to-right modeling while controlling the context length, we construct samples using a fixed-length (L) context window, if the context is shorter than L, it is left padded with <PAD>. We employ an LSTM because password segments exhibit strong sequential dependencies: the choice of the next segment often depends on both recent patterns (e.g., common affixes) and longer-range structure (e.g., base word + suffix + digits). Moreover, LSTM’s gating mechanism helps retain informative history while mitigating vanishing gradients, making it well-suited for modeling variable-length segment sequences under a fixed context window. The resulting input sequence is fed into a multi-layer LSTM, and a two-layer fully connected head outputs logits over the entire segment vocabulary. Model parameters are optimized using cross-entropy loss, with gradient clipping applied for stable training; the best checkpoint is selected according to the training loss.

For example, suppose the original password is “520love”, which is segmented into the segment sequence “520”, “love”. And the length L of context window is 3. During training, we formulate it as a next segment prediction task. Specifically, with <START> as the start symbol, the model predicts the first segment “520” given the context <START>. To obtain a fixed length input, we pad the left side with <PAD> because the context length is shorter than L, i.e., [<PAD>, <PAD>, <START>]. Then, the model predicts the next segment “love” given the context <START>, “520”. Finally, it predicts the termination symbol <END> given the context <START>, “520”, “love”, thereby completing the probabilistic modeling of the segment sequence.

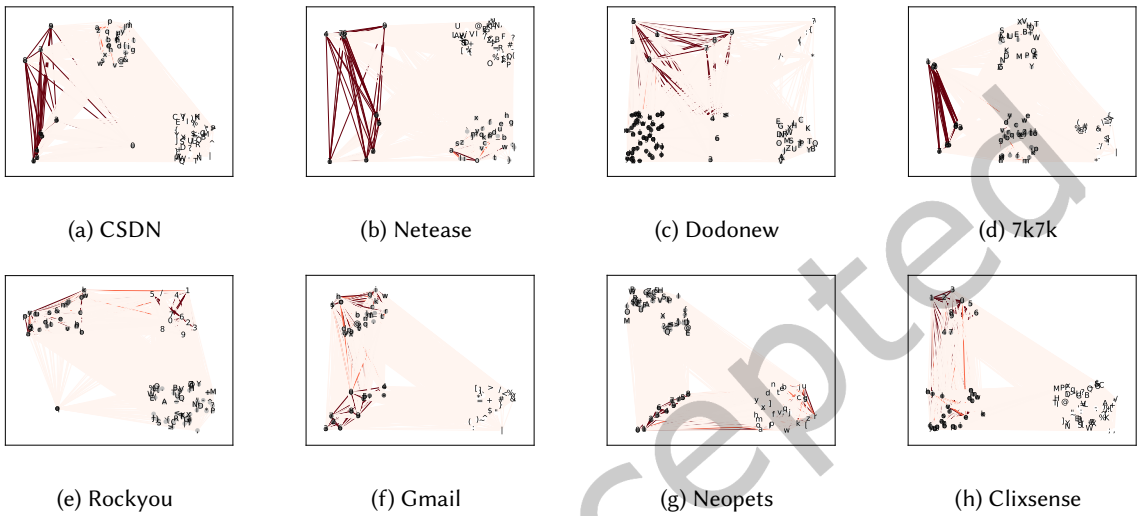
3.1.4 Evaluation and guessing number simulation. For evaluation, the probability of a segmented password in the test set is computed by accumulating the log-probabilities of each segment predicted by the LSTM under the same context-window mechanism, plus the log-probability of emitting <END>. To improve efficiency on large test sets, we implement a streaming batch inference strategy: all step-wise context windows across passwords are packed into batches, and the corresponding ground-truth next-token log-probabilities are gathered and summed back to each password.

Finally, to estimate guess numbers, we generate candidate passwords from the trained model using stochastic sampling: at each step, we sample the next segment from the softmax distribution until <END> is produced or a maximum length constraint is reached, and repeat the process to obtain a large number of samples for Monte Carlo estimation. For instance, N passwords and their corresponding probabilities are obtained through random sampling and designated as set A. At this point, the passwords in the training set are then sorted by decreasing probability to obtain set B. Next, a new set C is created, which contains M subsets, with M being of the same length as set B. The set A is then used to determine all the passwords in the training set, and the probability is the product of the probabilities of the segments. Cyclically determine all the passwords in set A. If their probability is between the x password and the $x + 1$ password in set B, then add the probability value to the $x + 1$ subset in set C. After completing the above operation then operate on each subset in set C. Take the inverse of the values in the individual subsets and add them together, then divide by the total number of samples N. The value obtained at this point is the number of simulated guesses for the passwords in the corresponding set B:

Just Accepted

$$G(password) = \frac{\sum C_i}{N} \tag{2}$$

where $G(Password)$ represents the number of guesses for the corresponding password in set B, and C_i represents the i subset of set C.



(a) CSDN (b) Netease (c) Dodonew (d) 7k7k

(e) Rockyou (f) Gmail (g) Neopets (h) Clixsense

Fig. 2. The result of using Louvain algorithm to classify the characters of the passwords in the training set. Characters of the same type are clustered together within the same group.

Just Accepted

3.2 Clustering and Segmentation Statistics

3.2.1 Analysis of Clustering Results. As shown in Figure 2, we visualized the clustering results for each dataset. Given the different frequencies of connections between different characters, this article sets two frequency thresholds (i.e. 5000 and 10000) to determine the color of the edge. As the frequency of the connection between characters increases, the color of the edge gradually transitions from light red to dark red.

When we delve into the clustering results of passwords on different datasets, a significant pattern emerges: numbers and letters in passwords tend to cluster in the same or similar clusters, in contrast to this clustering, the distribution of special characters is highly decentralized. Taking CSDN clustering results as an example, special characters are usually widely distributed in clusters of numbers and letters. Regarding the clustering characteristics of numbers and letters, we believe that this is because most passwords set by users are composed of numbers and letters [19]. For special characters, their clustering features are significantly different from numbers and letters. Given the strict password creation policies that many websites [24] have in place for security reasons (e.g., must include letters, numbers, and special characters), we can surmise that the distribution of special characters in passwords may have something to do with these policies. In the process of creating a password, a user may first determine a password base consisting of numbers and letters, and then unconsciously add special characters at the end or in the middle of the password to comply with the website's password policy. Therefore, the high dispersion of these special characters in the clustering results may be a reflection of users' unconscious behavior.

3.2.2 Analysis of Segmentation Results. We counted and sorted the occurrence times of segments in the password. Table 3 lists the occurrence times of Top-10 segments with length greater than 3 in different data sets.

Table 3. Top-10 Segments in each dataset after segmenting the passwords.

Rank	CSDN		Netease		Dodonew		7k7k	
1	123456	1,159	wang	1,936	wang	2,809	123456	1,011
2	wang	949	zhang	1,608	zhang	2,270	wang	956
3	zhang	696	123456	1,432	chen	1,862	love	757
4	chen	606	yang	1,121	yang	1,772	zhang	715
5	love	591	love	1,107	love	1,741	chen	619
6	yang	504	woaini	1,084	123456	1,514	yang	567
7	csdn	410	chen	1,061	5201314	1,236	5201314	428
8	5201314	382	5201314	949	woaini	1,197	2008	394
9	1234	360	1314	912	long	1,174	1314	347
10	2009	338	huang	658	xiao	1,129	huang	341
Rank	**Rockyou**		**Gmail**		**Neopets**		**Clixsense**	
1	love	1,007	love	770	love	982	123456	1,071
2	june	484	1234	671	1234	949	1234	911
3	angel	445	july	656	123456	915	clix	660
4	baby	426	march	579	123456789	785	love	632
5	july	403	123456	473	12345	709	june	596
6	1234	400	june	425	angel	602	clixsense	562
7	sexy	333	alex	393	neopets	547	july	521
8	pink	330	blue	345	ever	493	12345	508
9	alex	320	angel	345	monkey	452	april	469
10	april	319	april	344	pokemon	444	123456789	453

The statistical data reveal that ordered numeric segments appear frequently in passwords for both Chinese and English users. For example, “1234” appears in the Top-10 segments across all eight datasets. For Chinese users, they are more likely to include the pinyin of their surname in their passwords, such as “wang” or “zhang”. On the other hand, English users tend to prefer using months or other short, meaningful English words, such as “june” or “angel”. Additionally, we observed that uppercase letters and special characters are absent from the Top-10 segments.

Users commonly prefer simple and easy-to-remember weak password segments such as “123456” or “love”, which appear frequently across various platforms, reflecting a tendency to prioritize convenience over security. Additionally, users often embed personal information, such as their name in pinyin, birth year, or significant dates, into their passwords in pursuit of personalization, which significantly increases the predictability and vulnerability of these passwords. Based on the segmentation statistics obtained from different datasets, we further observe noticeable differences in password construction habits across user groups from different linguistic and cultural backgrounds. For example, English users tend to favor short English words and personal names, whereas Chinese users more frequently adopt combinations of pinyin, numbers, and ideographic expressions. It is important to note that these observations are derived solely from the statistical analysis of segmentation results and are not explicitly incorporated as prior knowledge or handcrafted features in the clustering or model training process. Instead, the proposed method remains entirely data-driven and learns segmentation patterns automatically from the underlying password distributions.

Just Accepted

Table 4. Comparison between CKL_PCFG and PassCluster.

Dimension	CKL_PCFG	PassCluster
Segmentation criterion	Merging frequent adjacent character pairs	Community detection on a global co-occurrence graph
Granularity control	Determined by the number of merges or vocabulary size	Adaptively determined by graph structure and modularity optimization
Modeling bias	Tends to favor locally frequent substrings	Emphasizes global structural consistency

Just Accepted

3.3 Comparison with CKL_PCFG

A closely related line of our work is CKL_PCFG. BPE constructs password chunks by repeatedly merging the most frequent adjacent character pairs in the training corpus. This strategy is simple and effective for identifying frequent substrings, and it demonstrates the usefulness of modeling passwords at a granularity between individual characters and coarse templates.

However, the proposed Louvain-based segmentation differs from BPE-based segmentation in several important aspects, as summarized in Table 4. First, BPE relies on greedy merging of high-frequency adjacent character pairs, whereas PassCluster constructs a global character co-occurrence graph and applies community detection to identify structural character groups. Second, the granularity of BPE-based segmentation is usually controlled by external parameters, such as the number of merge operations or the vocabulary size. In contrast, PassCluster determines character categories according to the graph structure and modularity optimization, making the segmentation process more adaptive to the underlying password distribution. Third, BPE may be biased toward locally frequent substrings, while PassCluster aims to preserve globally coherent structural relationships among password characters. To further illustrate the difference at the output level, Table 5 presents the top-10 frequent segments with length no less than four generated by CKL_PCFG and PassCluster on the RockYou dataset. As shown in the table, the high-frequency segments produced by BPE-based segmentation are mainly concentrated on consecutive digit strings and common local substrings, such as “123456”, “123456789”, “1234” and “2007”. This indicates that the greedy merging mechanism of BPE tends to preserve locally frequent adjacent character patterns in the training corpus, especially increasing numeric sequences and fixed high-frequency substrings. In contrast, the high-frequency segments produced by PassCluster exhibit stronger semantic or structural completeness, such as “love”, “june”, “angel”, “baby”, “pink” and “alex”. These segments usually correspond to names, months, emotional words or common semantic units in real-world passwords. This observation suggests that PassCluster can learn more stable and interpretable password components from the global character co-occurrence structure.

This distinction also affects the subsequent password modeling process. In CKL_PCFG, BPE-derived chunks are incorporated into a PCFG-style template framework. In contrast, PassCluster first learns latent character communities and then represents each password as a sequence of segments, which are modeled by an LSTM-based left-to-right probabilistic model. Therefore, PassCluster differs from prior BPE-based work not only in the segmentation algorithm, but also in how segmented password structures are represented and generated.

Overall, BPE-based segmentation confirms the value of chunk-level password modeling, while PassCluster further explores a graph-based and community-driven segmentation paradigm. The empirical comparison with CKL_PCFG in Section 4 further evaluates the effect of this difference under both intra-site and cross-site guessing scenarios.

Table 5. Top-10 segments generated by CKL_PCFG and PassCluster on the RockYou dataset.

Model	Top-1	Top-2	Top-3	Top-4	Top-5	Top-6	Top-7	Top-8	Top-9	Top-10
CKL_PCFG	123456	123456789	love	password	baby	ilove	iloveyou	1234	princess	2007
PassCluster	love	june	angel	baby	july	1234	sexy	pink	alex	april

4 Experimental evaluation

4.1 Experimental Setup

For this research, eight datasets were adopted, namely CSDN, Netease, Dodonew, 7k7k, RockYou, Gmail, Neopets and Clixsense. Among them, the first four are Chinese datasets, and the latter four are English datasets. Each dataset has its own unique password distribution characteristics, and the differences in password sets among different language regions are even more significant. For example, passwords set by English users are mostly alphabetic strings, while those of Chinese-speaking users are more likely to be pure numeric strings.

Based on the similarities and differences in the password distributions of the training dataset and the test dataset, we divided the experiments into intra-site and cross-site scenarios. In the intra-site attack scenario, due to the limitations of the dataset size, the CSDN training set randomly selects 900,000 passwords, and the test set randomly selects 100,000 passwords. For the other seven datasets, the training sets contain 1,800,000 passwords, and the test sets contain 200,000 passwords. Consistent with prior password modeling studies [26, 46], we adopted a 9:1 split between the training set and the test set. The larger training set helps the model capture more comprehensive password distribution patterns, while the test set still contains a sufficiently large number of passwords to ensure statistically reliable evaluation. This setting is based on the assumption that attackers can obtain the leaked passwords on the website and attempt to crack the remaining passwords. In reality, many websites have experienced password leakage incidents repeatedly, which makes this kind of intra-site guessing meaningful in practical scenarios.

Regarding the cross-site experiments, we assume that attackers have no prior knowledge of the target passwords. Specifically, the trained model is applied to one dataset (e.g., Rockyou) to attempt to crack the passwords in another dataset (e.g., Gmail, CSDN).

The experiments were carried out on a system running Ubuntu 22.04, featuring four NVIDIA Tesla V100 GPUs, an Intel(R) Xeon(R) Gold 6338 processor, and 256GB of RAM. The model is implemented using the Python programming language, with PyCharm serving as the primary development environment. The Louvain algorithm is implemented using the community library.

4.2 Compared Models

OMEN [9]. As an innovative password guessing model, OMEN demonstrates significant advantages in the field of cryptography. Based on the Markov model, it adopts a 4-grams analysis method, a character set of 94 printable ASCII characters excluding the space character, and an improved enumeration algorithm, enumPwd, enabling it to capture the characteristics of passwords more comprehensively and deeply. Its most prominent feature is the ability to generate candidate passwords according to the probability of password occurrence and output them in descending order of probability.

PCFG [28]. We adopted the most recent version of the PCFG model, specifically v4.6, configured with the recommended settings. This version enhances cracking performance by incorporating a broader range of password patterns, such as keyboard sequences, email formats, and website-related strings, making it more versatile compared to earlier iterations.

CKL_PCFG [44]. CKL_PCFG is a probability context-free grammar model based on password chunks. It uses the PwdSegment method to split passwords according to the set average length parameter (avg_len) to obtain a

Just Accepted

chunk vocabulary. When constructing templates, new labels such as L, U, D, S, DM, TM, FM, etc. are defined based on the type of chunks. The password probability is calculated by learning the template and chunk instantiation probability from the training set.

FLA [20]. FLA is an RNN-based password guessability estimation model that enables efficient and lightweight yet accurate password strength evaluation. In our implementation, we adopted an RNN consisting of two LSTM layers with 200 cells each, used Adam as the optimizer, and trained the model for 20 epochs.

RFGuess [39]. RfGuess is a random forest-based password guessing model that leverages classical machine learning to predict the next character in a password from its prefix. In our implementation, we built RFGuess with 30 decision trees, set the minimum number of leaf nodes to 10, used a maximum feature ratio of 80%, and kept the remaining hyperparameters as the default settings in scikit-learn.

JTR/Hashcat [29, 33]. For dictionary based attacks, we used both JTR and Hashcat, applying mangling rules to the training set (e.g., Rockyou). In JTR, the SpiderLabs mangling rules were utilized, while Hashcat employed the integrated Gen2 rule set. It is important to note that the guessing performance of these tools can vary significantly due to their differing approaches: JTR follows a word-major order, whereas Hashcat operates on a rule-major order.

PassGAN [16]. PassGAN is a deep generative password guessing model based on Generative Adversarial Networks. It replaces manually designed mangling rules with a data-driven generator that learns password structures directly from leaked password datasets. Specifically, PassGAN adopts the Improved Wasserstein GAN framework, where a generator produces password candidates from random noise and a discriminator distinguishes generated samples from real leaked passwords. In our implementation, we followed the original IWGAN-based setup, using residual convolutional blocks, the Adam optimizer, a batch size of 64, and 199,000 training iterations.

AdaMs_Hashcat [27]. AdaMs_Hashcat is an adaptive and dynamic mangling-rule based password guessing model designed to better approximate real-world dictionary attacks. It combines Adaptive Mangling Rules with dynamic guessing strategies, where each dictionary word is associated with a dedicated subset of rules selected at runtime by a neural network. During guessing, successfully cracked passwords are further used to augment the dictionary, and the rule budget is dynamically adjusted according to observed hits. In our implementation, AdaMs_Hashcat was built on the CPU legacy version of Hashcat, with a modified word-major workflow and a batch size of 4096 dictionary words.

PassGPT [31]. PassGPT is an autoregressive Transformer-based password generation model built upon the GPT-2 architecture. It models the probability of each character conditioned on all previous characters and sequentially samples passwords from the learned distribution. Unlike GAN-based models that generate passwords as whole strings, PassGPT explicitly estimates the password probability distribution and enables character-level guided generation under arbitrary constraints. In our implementation, PassGPT used a 256-character ASCII vocabulary and generated candidate passwords by autoregressively predicting the next token.

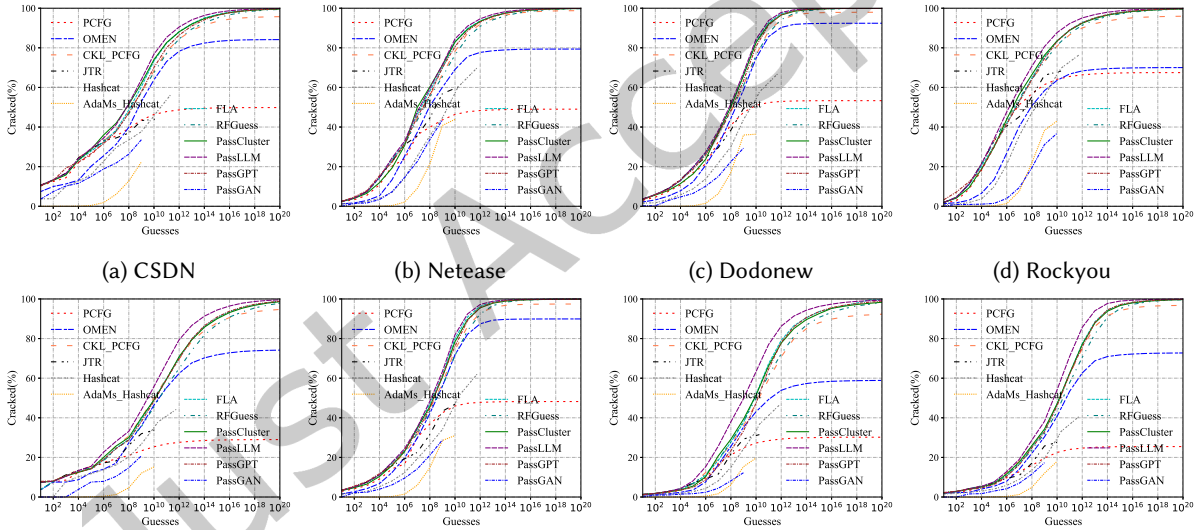
PassLLM [48]. PassLLM is a large-language-model-based password guessing framework that introduces parameter-efficient fine-tuning and task-specific generation algorithms for trawling and targeted attacks. We use Mistral-7B-v0.1 as the foundation model and reproduce the original non-distilled trawling pipeline, including LoRA fine-tuning, task-specific password generation and Monte Carlo guess-number estimation. The model is trained for three epochs using AdamW in fp16 precision, with a batch size of 32, gradient accumulation over 8 steps, and 4 GPUs in data-parallel (an effective batch size of 4,096), a learning rate of 5×10^{-4}, weight decay of 0.01, a linear warmup ratio of 0.2, LoRA rank 16, LoRA alpha 64, LoRA dropout 0.3, and the query, key and value projection modules as LoRA targets. The LoRA adapter contains approximately 9.44M trainable parameters and occupies about 36MB, while the merged model used for inference occupies approximately 14GB.

Just Accepted

4.3 Comparison of Cracking Rates

We compared PassCluster with a series of representative baseline models under both intra-site and cross-site password guessing scenarios. The overall results are presented in Figure 3 and Table 6, including four intra-site datasets and four cross-site evaluation settings. PassCluster achieves competitive guessing performance and outperforms most traditional statistical, rule-based, and machine-learning-based approaches, but it performs slightly worse than FLA and PassGPT in several settings. This difference mainly arises because these character-level models directly preserve fine-grained character dependencies and can fit frequent short patterns and local variations more effectively, whereas PassCluster introduces a segment-level structural bias that may make such localized patterns less explicit. PassCluster also performs slightly worse than PassLLM across the eight reported settings. A likely reason is that PassLLM benefits from a substantially larger pretrained foundation model and a task-specific generation pipeline, providing greater capacity to learn diverse and long-range password patterns. However, compared with these models, the main comparative advantages of PassCluster are its considerably smaller storage footprint and substantially higher generation throughput.

Based on the overall experimental results, we further selected seven representative baseline models that are highly related to PassCluster for more detailed comparative analysis and visualization in the following subsections. These models cover different categories of password guessing approaches, including probabilistic models, deep learning based methods and rule-based generation techniques, enabling a more comprehensive evaluation of the characteristics and effectiveness of PassCluster.



(a) CSDN (b) Netease (c) Dodonew (d) Rockyou

(e) Rockyou2CSDN (f) Netease2Dodonew (g) Dodonew2Rockyou (h) Rockyou2Dodonew

Fig. 3. Cracking performance of PassCluster and all compared state-of-the-art models.

4.3.1 Intra-site Attack. Figure 4 shows that the PassCluster model significantly outperforms most existing models in terms of cracking rate. With 10^{20} guesses, the PassCluster model outperforms PCFG by an average of 64.9% (from 30.6% to 103.7%). It outperforms OMEN by an average of 16.8% (from 7.2% to 42.3%). Against JTR, the average improvement is 63.9% (from 30.7% to 126.7%). When compared with Hashcat, the average improvement is 40.8% (from 22.3% to 76.9%). And over CKL_PCFG, the average improvement is 2.3% (from 0.1% to 4.0%). It

Just Accepted

Table 6. Password guessing performance of PassCluster and baseline models under a guessing budget of 10^{20}. The table reports the number of successfully guessed passwords on four intra-site and four cross-site scenarios, including Rockyou→CSDN (CrossSce1), Netease→Dodonew (CrossSce2), Dodonew→Rockyou (CrossSce3), and Rockyou→Dodonew (CrossSce4). Since Hashcat, JTR, AdaMs_Hashcat, and PassGAN are generation based approaches that do not support probability-ordered enumeration, their practical maximum number of generated guesses is limited to approximately 10^{10}.

Model	Numbers of Cracked Password in Diffrent Scenarios							
	CSDN	Netease	Dodonew	Rockyou	CrossSce1	CrossSce2	CrossSce3	CrossSce4
OMEN	84147	158727	184807	140031	74169	179826	117738	145461
PCFG	49830	98045	106642	134962	29000	96423	60394	50941
CKL_PCFG	95731	197535	195911	191901	94714	194899	184628	193507
FLA	99664	199762	199954	199323	98806	199816	199108	199513
RFGuess	99414	199549	199856	199179	97847	199786	196086	199232
PassGAN	33603	87174	58669	73694	20787	57689	24522	34682
AdaMs_Hashcat	22385	87761	72802	86115	15207	62076	39400	36018
PassGPT	99666	199810	199916	199446	98920	199818	197774	199525
PassLLM	99874	199900	199965	199735	99566	199884	198840	199867
JTR	43930	120346	107415	136555	34292	95344	63029	56391
Hashcat	56283	139894	135129	153008	44222	124186	92508	81538
PassCluster	99592	199684	199876	199288	98622	199792	196680	199395

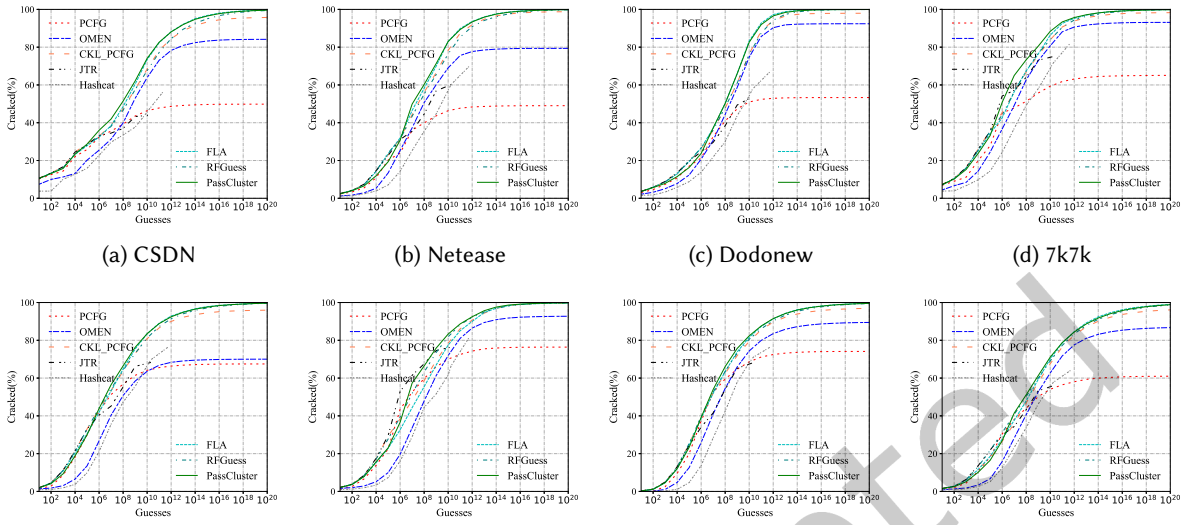
outperforms RFGuess by an average of 0.1% (from 0.1% to 0.2%). Compared with FLA, PassCluster exhibits an average performance change of -0.04% (from -0.1% to 0.1%). However, when the number of guesses ranges from 10^7 to 10^{10}, PassCluster demonstrates a more pronounced advantage over both FLA and RFGuess: it achieves an average improvement of 5.5% compared with FLA and 6.0% compared with RFGuess.

4.3.2 Cross-site Attack. Figure 5 presents the guessing results for the cross-site attack scenario. Significantly, the PassCluster model outperforms most of the state-of-the-art models, including PCFG, OMEN, JTR, Hashcat, and CKL_PCFG, RFGuess in both same language and different language crosssite scenarios. When the number of guesses is set to 10^{20}, the PassCluster model outperforms PCFG by an average of 222.9% (from 115.9% to 306.0%). Against OMEN, it shows an average improvement of 55.1% (from 14.3% to 85.5%). Compared to JTR, the average improvement is 196.6% (from 70.8% to 293.9%), and over Hashcat it averages a 116.9% improvement (from 40.3% to 168.7%). In comparison with CKL_PCFG, the PassCluster model achieves an average improvement of 6.3% (from 3.9% to 11.9%) in the cracking rate. PassCluster outperforms RFGuess by an average of 0.9% (from 0.6% to 1.4%). Compared with FLA, it exhibits an average performance change of -0.5% (from -1.9% to 0.2%). In the 10^7 to 10^{10} guesses range, PassCluster demonstrates clear superiority, achieving a 6.5% (from 5.2% to 8.9%) improvement over RFGuess and a 5.2% (from 2.3% to 9.5%) improvement over FLA.

4.3.3 Performance Analysis and Discussion. In the same experimental setting, the PassCluster outperforms most of competitors in terms of cracking rate improvement. It showcases a remarkable advantage in the cross-site password guessing task.

In the 10^{20} guesses, PassCluster does not exhibit a clear advantage over RFGuess and FLA, and may even perform slightly worse. A plausible explanation is that passwords covered at this scale are often composed of multiple semantically meaningless segments, which implies a much larger and more dispersed combinatorial space. As a segmentation-based cracking model, PassCluster has to decompose such passwords into more segments

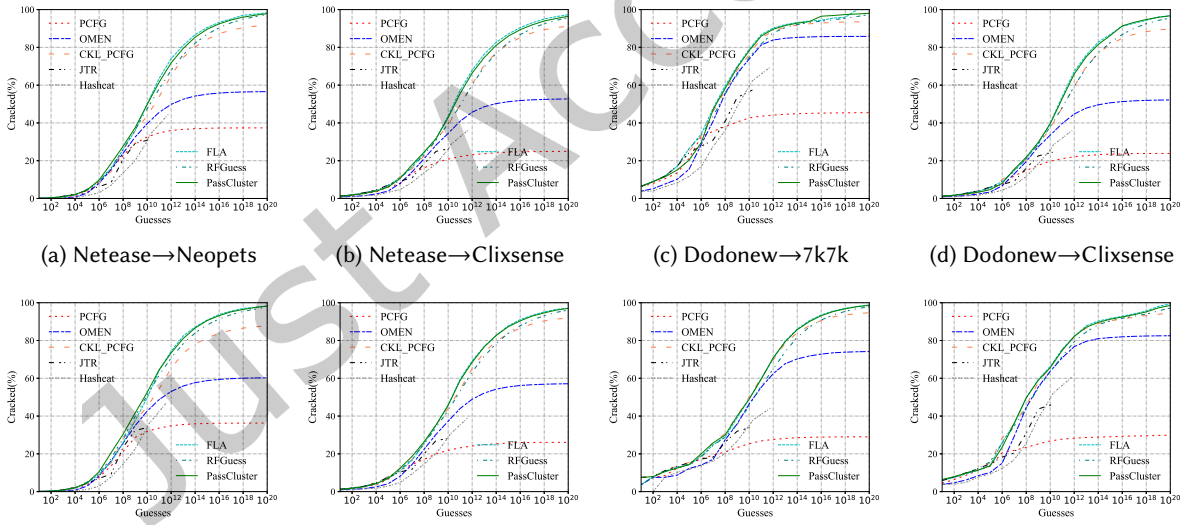
Just Accepted



(a) CSDN (b) Netease (c) Dodonew (d) 7k7k

(e) Rockyou (f) Gmail (g) Neopets (h) Clixsense

Fig. 4. Cracking performance of PassCluster and other different types of state-of-the-art models in intra-site guessing scenarios.



(a) Netease→Neopets (b) Netease→Clixsense (c) Dodonew→7k7k (d) Dodonew→Clixsense

(e) 7k7k→Neopets (f) 7k7k→Clixsense (g) Rockyou→CSDN (h) Clixsense→7k7k

Fig. 5. Cracking performance of PassCluster and other different types of state-of-the-art models in cross-site guessing scenarios.

Just Accepted

and explore many more segment combinations before it can match them. Consequently, probability mass is spread across a larger number of candidates, leading to lower estimated probabilities for individual passwords and weakening PassCluster's ranking performance under this extremely large guess budget. Notably, on the Gmail dataset, PassCluster outperforms FLA under nearly all guess limits. A plausible explanation is that Gmail passwords exhibit stronger human-memorable, segment-consistent structures (e.g., word stems followed by digits/symbols), which aligns well with PassCluster's segmentation and left-to-right sequence modeling; in contrast, the other leaks contain more noisy, weakly semantic, and heavily transformed strings that induce finer and less stable segmentation, enlarging the segment-combination space and diluting probability mass, thereby favoring models like FLA. By contrast, when the number of guesses falls within 10^7 to 10^{10}, passwords are more likely to consist of a few meaningful and reusable segments. In this setting, PassCluster can better exploit segment-level regularities to generate and prioritize candidates, making it more effective under a limited guess budget and yielding a more pronounced advantage. From a practical perspective, real-world guessing typically uses on the order of 10^{10} candidates for actual attempts. A guess budget of 10^{20} is excessively large for password cracking in practice, incurring prohibitive time and computational costs and thus offering limited real-world relevance.

4.4 Analysis

4.4.1 Efficiency Comparison of Different Models. To provide a comprehensive analysis of the training time, model size, and password generation speed, we calculated the average training time, model size, and generation speed for each model across all datasets (see Table 7). It is worth noting that since JTR and Hashcat directly use pre-determined rules, they do not require training.

Table 7. The efficiency of PassCluster and other state-of-the art models.

Models	Model Size	Training Time	Generated PW/s
PCFG	37.8 M	408.3 s	207,912.1
OMEN	10,035.2 M	5.3 s	1,679,402.1
JTR	115.0 M	N/A	1,907,159.5
Hashcat	65.0 M	N/A	1,037,250.2
CKL_PCFG	59.6 M	19,291.1 s	302,793.1
FLA	5.5 M	103,010.1 s	3625.2
RFGuess	1,331.2 M	624.3 s	93.6
PassGAN	14.9 M	42,340 s	48,971
AdaMs_Hashcat	756.0 M	192,600.4 s	78,082.3
PassGPT	219.6 M	7,560.2 s	1,037.2
PassLLM	14,372.0 M	27,240.0 s	155.0
PassCluster	418.0 M	2,475.2 s	6158.0

The model size of PassCluster is 418.0M, which is larger than some probabilistic models such as PCFG (37.8M), FLA (5.5M) and PassGAN (14.9M), but remains considerably smaller than OMEN (10,035.2M) ,RFGuess (1,331.2M), AdaMs_Hashcat (756.0M) and PassLLM (14,372.0M). Compared with lightweight statistical models, the relatively larger storage overhead of PassCluster mainly comes from its segment-based probabilistic modeling strategy. Specifically, PassCluster needs to store a large number of password segments together with their corresponding conditional probability distributions and transition relationships. Since the model performs generation based on

Just Accepted

segment-level probabilistic transitions, these statistical structures must be retained to support accurate probability lookup and sampling during password generation.

In terms of efficiency, PassCluster requires 2,475.2 seconds for training. This training time is longer than OMEN (5.3 s), PCFG (408.3 s) and RFGuess (624.3 s), but remains substantially shorter than CKL_PCFG (19,291.1 s), FLA (103,010.1 s), AdaMs_Hashcat (192,600.4 s) and PassLLM (27,240.0 s). Compared with recent deep learning and large language model based approaches, PassCluster achieves a better balance between training cost and modeling capability.

PassCluster achieves a password generation speed of 6,158.0 PW/s. Its generation throughput is lower than highly optimized rule-based or lightweight probabilistic approaches such as OMEN, JTR, Hashcat, PCFG, and CKL_PCFG, but remains significantly faster than PassLLM (155.0 PW/s) and RFGuess (93.6 PW/s). It is worth noting that PassCluster significantly outperforms FLA (3,625.2 PW/s) and PassGPT (1,037.2 PW/s) in generation efficiency. As shown in Table 8, we further compare PassCluster with several mainstream neural-network-based password guessing models (e.g., FLA, PassGPT, and PassLLM) in terms of the number of passwords cracked per minute. Specifically, under a guessing budget of 10^8, we report the password cracking efficiency of each model across four intra-site scenarios, including Gmail, 7k7k, Neopets, Rockyou. Under this equal guess budget, PassLLM cracks more passwords than PassCluster on Gmail and Rockyou, whereas PassCluster cracks more passwords on 7k7k and Neopets. Neither model consistently achieves a higher cracking count across all four scenarios. However, PassCluster cracks substantially more passwords per minute in all four scenarios because of its higher generation throughput, demonstrating substantially better time-normalized efficiency under our hardware and implementation settings. The relatively lower throughput of PassCluster compared with lightweight methods is mainly due to its segment-level generation mechanism. During password generation, PassCluster starts from an initial state and repeatedly samples the next segment according to the learned conditional probability distributions until a termination symbol is reached. This iterative probability lookup and transition sampling process introduces additional computational overhead, especially when the segment vocabulary and transition relationships become large. Nevertheless, PassCluster still maintains practical generation efficiency while providing stronger expressive capability than many traditional statistical models.

Table 8. Comparison of password cracking efficiency between PassCluster and neural-network-based password guessing models under a 10^8 guessing budget.

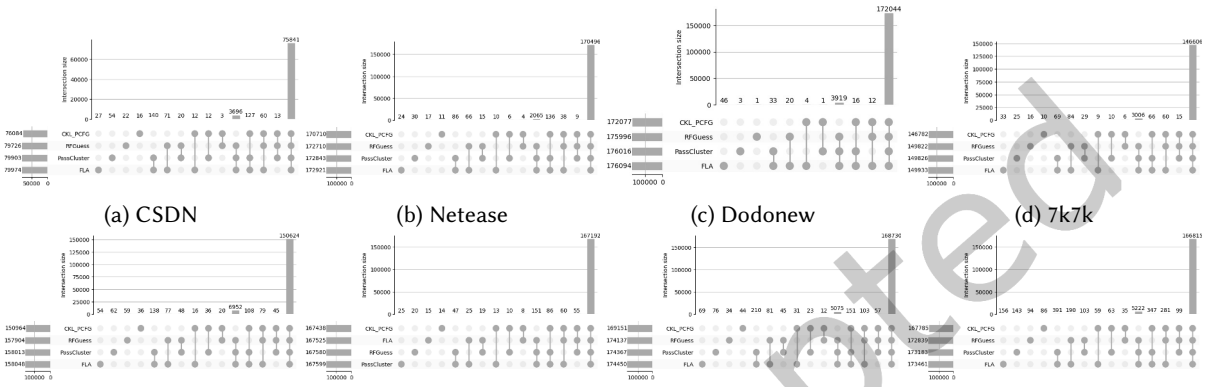
Model	Gmail		7k7k		Neopets		Rockyou	
	Numbers	Cracked/min	Numbers	Cracked/min	Numbers	Cracked/min	Numbers	Cracked/min
FLA	110854	241.11	134833	293.24	126884	275.99	129440	281.55
PassGPT	110566	68.80	131515	81.84	125948	78.38	128230	79.78
PassLLM	138149	12.85	138844	12.91	127641	11.87	141384	13.15
PassCluster	132860	490.89	146643	541.82	132193	488.43	132911	491.08

4.4.2 Overlap. In the above experiments, the datasets contained duplicate passwords, meaning simple passwords like "123456" were counted multiple times. However, in real-world cracking scenarios, there is no need to repeatedly attempt the same password. Thus, for an efficient model, the number of unique passwords it cracks more accurately reflects its cracking efficiency. Additionally, the number of passwords a model can crack that other models fail to crack also demonstrates its capability to handle more complex passwords.

We compared the unique passwords cracked by PassCluster with those cracked by CKL_PCFG, FLA, and RFGuess, under 10^{20} guesses (as shown in Figure 6). The results indicate that PassCluster achieves a higher number of unique cracked passwords in the vast majority of scenarios and is also capable of cracking passwords

Just Accepted

that other models fail to crack. For example, on the CSDN dataset, PassCluster successfully cracked 54 unique passwords, significantly outperforming CKL_PCFG (16), FLA (27), and RFGuess (22). This shows that, under the same cracking conditions, PassCluster exhibits higher cracking efficiency. This superior performance is likely due to the fact that PassCluster balances semantic meaning and character connectivity. By segmenting passwords into logical units, the model captures intricate patterns that other models miss, enabling it to crack a broader range of unique and complex passwords with higher efficiency.



(a) CSDN (b) Netease (c) Dodonew (d) 7k7k

(e) Rockyou (f) Gmail (g) Neopets (h) Clixsense

Fig. 6. Overlap of PassCluster, CKL_PCFG, FLA and RFGuess for the guesses of 10^{20}. The horizontal bar chart on the left shows the number of unique passwords that each model can crack. The scatter plot and bar chart on the right illustrate the number of passwords that other models fail to crack, but which the selected models can crack.

Table 9. Unsafe-error analysis between PassCluster and existing PSMs. Rows correspond to the strength intervals estimated by PassCluster, while columns correspond to those predicted by the comparison models. Diagonal entries (green) represent consistent estimations. Entries below the diagonal (red) indicate unsafe errors where comparison models overestimate password strength, while entries above the diagonal (purple) indicate cases where PassCluster gives stronger estimations. The color intensity reflects the relative proportion of passwords within each region.

FLA	PassCluster						CKL_PCFG	PassCluster					
	$> 10^0$	$> 10^2$	$> 10^5$	$> 10^8$	$> 10^{11}$	$> 10^{14}$		$> 10^0$	$> 10^2$	$> 10^5$	$> 10^8$	$> 10^{11}$	$> 10^{14}$
$> 10^0$	73	15	0	0	0	0	$> 10^0$	78	18	0	0	0	0
$> 10^2$	27	15571	2473	42	4	0	$> 10^2$	21	19782	5289	2046	441	78
$> 10^5$	0	7611	47211	6560	102	20	$> 10^5$	0	4092	45637	7714	272	26
$> 10^8$	0	658	7971	44917	4843	226	$> 10^8$	0	10	6873	41705	5708	415
$> 10^{11}$	0	43	153	3503	18193	1750	$> 10^{11}$	0	0	14	3581	16127	1809
$> 10^{14}$	0	3	3	33	1142	4639	$> 10^{14}$	0	0	0	15	1723	4149

4.5 Password Strength Meter

Given that our model demonstrates superior performance in cracking passwords, we have designed a PSM to help users select stronger passwords. Figure 7 shows the interface of this PSM. When a user enters and confirms

Just Accepted

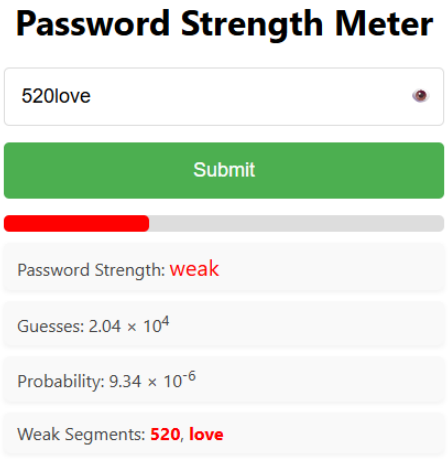


Fig. 7. The interface of PSM.

their password, the PSM provides feedback on the password's probability, guess count, and strength based on PassCluster. According to [11], when the guessing number is less than 10^6, the password is classified as weak password; when the guessing number is between 10^6 and 10^{14}, the password is classified as moderate password; and when the guessing number exceeds 10^{14}, the password is considered strong password. Additionally, due to the model's segmentation feature, this PSM evaluates the strength of each segment in the password. If any of the segments are weak, the PSM will prompt the user to change those segments (e.g., in the password "520love", both "520" and "love" are weak segments).

Table 9 reports the unsafe errors of different PSMs. We compare PassCluster PSM with existing well-performing PSMs based on FLA and CKL_PCFG. In the confusion matrices, rows denote the strength intervals estimated by PassCluster, while columns denote those predicted by the comparison model. Therefore, entries below the diagonal indicate cases where the comparison model overestimates password strength, and these entries are highlighted in red. By contrast, entries above the diagonal indicate cases where PassCluster gives a stronger estimation than the comparison model, highlighted in purple. Diagonal entries, highlighted in green, represent consistent estimations between PassCluster and the comparison model. As shown in Table 9, most passwords are concentrated around the diagonal, suggesting that PassCluster generally produce estimations close to the comparison models. This confirms that when estimating the strength of password, our PSM is comparable to existing mainstream PSMs.

5 Conclusion and Future Work

This paper proposes a more efficient password cracking model, PassCluster, to address the security challenges of password-based authentication systems. The model introduces an innovative password segmentation method that successfully balances the semantic and character connectivity issues present in existing segmentation approaches. Through comprehensive evaluations, PassCluster demonstrates excellent cracking capabilities in both intra-site and cross-site scenarios. Additionally, we conducted a series of experimental analyses to prove that PassCluster performs exceptionally well in cracking complex passwords. Finally, a PSM based on this model is developed to assist users in creating stronger passwords and enhancing overall password security.

Just Accepted

Although the PassCluster model demonstrates significant advantages in password guessing accuracy and efficiency, there remains considerable room for further extension and optimization. First, although this study focuses on integrating Louvain-based segmentation with an LSTM sequence model, the proposed segmentation strategy is not inherently tied to LSTM. It can be viewed as a data-driven tokenization module that transforms raw character-level passwords into segment-level sequences before probabilistic or neural modeling. This design suggests potential compatibility with other password modeling frameworks. For example, in Markov-based models, the learned segments could replace individual characters as transition units; in PCFG-based models, they could refine terminal symbols or component dictionaries; and in Transformer- or GPT-based password models, they could serve as password-specific tokens that provide more meaningful input units than raw characters. However, the effectiveness of such integrations may vary across model architectures and datasets, and a systematic evaluation of Louvain-based segmentation combined with Markov, PCFG, and GPT-style models will be an important direction for future work. Second, while the model shows improved generalization across multilingual and cross-cultural datasets, further enhancements are possible through the incorporation of multilingual semantic information and cultural features to improve tokenization and clustering accuracy. Lastly, although the existing PSM provides basic feedback functionality, its effectiveness could be enhanced by integrating it with user behavior modeling systems to deliver dynamic, personalized security recommendations, thereby promoting better password practices and strengthening overall system security.

Just Accepted

References

[1] 7k7k. 2011. https://www.7k7k.com.

[2] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment* 2008, 10 (2008), P10008.

[3] Haibo Cheng, Wenting Li, Ping Wang, and Kaitai Liang. 2021. Improved probabilistic context-free grammars for passwords using word extraction. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2690–2694.

[4] Clixsense. 2016. https://www.ysense.com.

[5] CSDN. 2011. https://www.csdn.net/.

[6] Anupam Das, Joseph Bonneau, Matthew Caesar, Nikita Borisov, and XiaoFeng Wang. 2014. The tangled web of password reuse.. In *NDSS*, Vol. 14. 23–26.

[7] Dodonew. 2011. https://www.dodonew.com/.

[8] Mingjian Duan, Ming Xu, Shenghao Zhang, and Weili Han. 2026. MoPE: A Mixture of Password Experts for Improving Password Guessing. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE.

[9] Markus Dürmuth, Fabian Angelstorf, Claude Castelluccia, Daniele Perito, and Abdelberi Chaabane. 2015. OMEN: Faster password guessing using an ordered markov enumerator. In *Engineering Secure Software and Systems: 7th International Symposium, ESSoS 2015, Milan, Italy, March 4-6, 2015. Proceedings 7*. Springer, 119–132.

[10] Enzoic. 2017. Can Passwords Really Be Replaced? What Makes Them Unique. https://www.enzoic.com/blog/can-passwords-really-replaced/. Accessed: 2026-09-26.

[11] Dinei Florêncio, Cormac Herley, and Paul C Van Oorschot. 2016. Pushing on string: The'don't care'region of password strength. *Commun. ACM* 59, 11 (2016), 66–74.

[12] Santo Fortunato. 2010. Community detection in graphs. *Physics reports* 486, 3-5 (2010), 75–174.

[13] Gmail. 2014. https://mail.google.com.

[14] Maximilian Golla, David Cash, and Blase Ur. [n. d.]. Reasoning Analytically About Password-Cracking Software. ([n. d.]).

[15] Albert Guan and Chia-Mei Chen. 2022. A Novel Verification Scheme to Resist Online Password Guessing Attacks. *IEEE Transactions on Dependable and Secure Computing* 19, 6 (2022), 4285–4293. doi:10.1109/TDSC.2022.3174576

[16] Briland Hitaj, Paolo Gasti, Giuseppe Ateniese, and Fernando Perez-Cruz. 2019. Passgan: A deep learning approach for password guessing. In *Applied Cryptography and Network Security: 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings 17*. Springer, 217–237.

[17] Shiva Houshmand, Sudhir Aggarwal, and Randy Flood. 2015. Next gen PCFG password cracking. *IEEE Transactions on Information Forensics and Security* 10, 8 (2015), 1776–1791.

[18] Andrea Lancichinetti and Santo Fortunato. 2009. Community detection algorithms: a comparative analysis. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 80, 5 (2009), 056117.

[19] Jerry Ma, Weining Yang, Min Luo, and Ninghui Li. 2014. A study of probabilistic password models. In *2014 IEEE Symposium on Security and Privacy*. IEEE, 689–704.
[20] William Melicher, Blase Ur, Sean M Segreti, Saranga Komanduri, Lujo Bauer, Nicolas Christin, and Lorrie Faith Cranor. 2016. Fast, lean, and accurate: Modeling password guessability using neural networks. In *25th USENIX Security Symposium (USENIX Security 16)*. 175–191.
[21] Arvind Narayanan and Vitaly Shmatikov. 2005. Fast dictionary attacks on passwords using time-space tradeoff. In *Proceedings of the 12th ACM conference on Computer and communications security*. 364–372.
[22] Neopets. 2016. https://www.neopets.com.
[23] Netease. 2015. https://www.163.com/.
[24] Nist and Emmanuel Aroms. 2012. NIST Special Publication 800-63 Electronic Authentication Guideline.
[25] Giulio Pagnotta, Dorjan Hitaj, Fabio De Gaspari, and Luigi V Mancini. 2022. Passflow: guessing passwords with generative flows. In *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 251–262.
[26] Dario Pasquini, Giuseppe Ateniese, and Carmela Troncoso. 2024. Universal neural-cracking-machines: Self-configurable password models from auxiliary data. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1365–1384.
[27] Dario Pasquini, Marco Cianfriglia, Giuseppe Ateniese, and Massimo Bernaschi. 2021. Reducing bias in modeling real-world password strength via deep learning and dynamic dictionaries. In *30th USENIX Security Symposium (USENIX Security 21)*. 821–838.
[28] PCFG. 2023. .
[29] Alexander Peslyak. 1996. *John the Ripper*.
[30] Madison Pickering, Garrison Hinson-Hasty, Luca Dovichi, Helena Williams, Nathaniel Kim, Aybala Esmer, and Blase Ur. 2026. Can Foundation LLMs Accurately Estimate Password Strength and Provide Appropriate Password Feedback?. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE.
[31] Javier Rando, Fernando Perez-Cruz, and Briland Hitaj. 2023. Passgpt: Password modeling and (guided) generation with large language models. In *European Symposium on Research in Computer Security*. Springer, 164–183.
[32] Rockyou. 2009. https://www.rockyou.com/.
[33] Jens Steube. 2009. *Hashcat*.
[34] Blase Ur, Patrick Gage Kelley, Saranga Komanduri, Joel Lee, Michael Maass, Michelle L Mazurek, Timothy Passaro, Richard Shay, Timothy Vidas, Lujo Bauer, et al. 2012. How does your password measure up? The effect of strength meters on password creation. In *21st USENIX security symposium (USENIX Security 12)*. 65–80.
[35] Rafael Veras, Christopher Collins, and Julie Thorpe. 2014. On semantic patterns of passwords and their security impact.. In *NDSS*. Citeseer.
[36] Verizon Business. 2025. *2025 Data Breach Investigations Report*. Technical Report. Verizon. https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf
[37] Chenyang Wang, Fan Shi, Min Zhang, Chengxi Xu, Miao Hu, Pengfei Xue, Shasha Guo, and Jinghua Zheng. 2026. CoT-DPG: A Co-Training Based Dynamic Password Guessing Method. In *Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2026.230755
[38] Ding Wang, Haibo Cheng, Ping Wang, Xinyi Huang, and Gaopeng Jian. 2017. Zipf's law in passwords. *IEEE Transactions on Information Forensics and Security* 12, 11 (2017), 2776–2791.
[39] Ding Wang, Yunkai Zou, Zijian Zhang, and Kedong Xiu. 2023. Password guessing using random forest. In *32nd USENIX Security Symposium (USENIX Security 23)*. 965–982.
[40] Yangde Wang, Weidong Qiu, Weicheng Zhang, Hao Tian, and Shujun Li. 2023. SE# PCFG: Semantically Enhanced PCFG for Password Analysis and Cracking. *arXiv preprint arXiv:2306.06824* (2023).
[41] Matt Weir, Sudhir Aggarwal, Michael Collins, and Henry Stern. 2010. Testing metrics for password creation policies by attacking large sets of revealed passwords. In *Proceedings of the 17th ACM conference on Computer and communications security*. 162–175.
[42] Matt Weir, Sudhir Aggarwal, Breno De Medeiros, and Bill Glodek. 2009. Password cracking using probabilistic context-free grammars. In *2009 30th IEEE symposium on security and privacy*. IEEE, 391–405.
[43] Jiahong Xie, Haibo Cheng, Rong Zhu, Ping Wang, and Kaitai Liang. 2022. WordMarkov: A New Password Probability Model of Semantics. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 3034–3038.
[44] Ming Xu, Chuanwang Wang, Jitao Yu, Junjie Zhang, Kai Zhang, and Weili Han. 2021. Chunk-level password guessing: Towards modeling refined password composition representations. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 5–20.
[45] Kunyu Yang, Xuexian Hu, Qihui Zhang, Jianghong Wei, and Wenfen Liu. 2022. VAEPass: A lightweight passwords guessing model based on variational auto-encoder. *Computers & Security* 114 (2022), 102587.
[46] Tao Yang and Ding Wang. 2025. Rankguess: Password guessing using adversarial ranking. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 682–700.
[47] Verena Zimmermann. 2021. *From the Quest to Replace Passwords towards Supporting Secure and Usable Password Creation*. Ph. D. Dissertation. Technical University of Darmstadt. doi:10.26083/tuprints-00017425

Just Accepted

[48] Yunkai Zou, Maoxiang An, and Ding Wang. 2025. Password guessing using large language models. In *34th USENIX Security Symposium (USENIX Security 25)*. 7799–7818.

[49] Yunkai Zou, Ding Wang, and Fei Duan. 2026. Success Rates Doubled with Only One Character: Mask Password Guessing. In *Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2026.241059

Received 1 March 2026; revised 3 October 2026; accepted 6 October 2026

Just Accepted